

VITTORIA HOME

Полное руководство: работа, менеджеры, деплой, публикация в магазинах

Сгенерировано из docs/*.md

Руководство по работе с приложением

VITTORIA HOME — руководство по работе с приложением

Простыми словами: что это, как связано, как пользоваться, как запустить и что нужно для «боевого» режима.

1. Что это за система

Это сервис для клиентов мебельной компании. Он состоит из четырёх частей:

1. **Backend (сервер)** — «мозг» системы (NestJS). Хранит данные, общается с amoCRM, отправляет SMS и уведомления.
2. **Веб-панель** (admin + кабинет партнёра) — сайт для сотрудников. Открывается в браузере.
3. **Мобильные приложения** для клиентов — iOS и Android (код написан; собираются на Mac / в Android Studio).
4. **Интеграция с amoCRM** — `vittoriaamo.amocrm.ru`. amoCRM — источник истины по сделкам.

Главная идея: менеджеры ведут сделки в amoCRM как обычно, а приложение показывает клиенту этап и % готовности его заказа и даёт чат с сервисом.

2. Как всё связано (важно понять)

Менеджер в amoCRM двигает сделку по воронке «VITTORIA HOME»

| (этапы = стадии готовности заказа)



Backend узнаёт об изменении → обновляет заказ у клиента



Клиент в приложении видит новый этап и % + получает уведомление

- **Воронка в amoCRM: «VITTORIA HOME»** с 7 этапами (см. §8). Менеджер просто перетаскивает сделку на нужный этап — приложение подхватит это автоматически (этап берётся из статуса сделки в воронке).
- Дополнительные данные (% готовности, комментарий, предоплата) хранятся в служебных полях сделки `vittoria_*` (их создал я, см. §8).

3. Веб-панель администратора

Адрес (локально): `http://localhost:5173`

Вход: по номеру телефона + код из SMS (в режиме разработки код виден в консоли сервера; в боевом — приходит реальной SMS).

Разделы (левое меню):

1. **Заказы** — список всех заказов: договор, изделие, этап, % готовности. Поиск и фильтр по этапу. Клик по строке → карточка заказа.
2. **Карточка заказа** — данные сделки + редактируемые поля: этап, % готовности, комментарий. Кнопка «Сохранить» обновляет заказ (и, в боевом режиме, синхронизируется с amoCRM) и уведомляет клиента.
3. **Чат** — переписка с клиентами: слева список диалогов (с бейджем непрочитанных), справа выбранный диалог + поле ответа. Обновляется автоматически (~10 сек). При открытии диалога непрочитанные гаснут.
4. **Партнёры** — список партнёров-дизайнеров; кнопка «Создать партнёра».
5. **Комиссии** — вознаграждения партнёров: список, фильтр по статусу, создание, смена статуса (Ожидает → Одобрено → Выплачено).
6. **Аудит** — журнал действий (кто что менял), с фильтром и просмотром «было/стало».
7. **Шаблоны уведомлений** — тексты push/SMS, редактируются без перевыпуска приложения.

4. Кабинет партнёра (роль «partner»)

Тот же сайт, но партнёр входит под своей ролью и видит только своё:

1. **Мои заказы** — заказы, где он привязан как партнёр (только просмотр).
2. **Карточка заказа** — данные без редактирования.
3. **Мои вознаграждения** — его комиссии с фильтром по статусу.
4. **Профиль** — имя, телефон; редактирование имени; выход.

Партнёр **не видит** чужих клиентов, не пишет в клиентские чаты и не меняет статусы.

5. Мобильное приложение клиента (iOS + Android)

Экраны:

1. **Вход** — телефон → код из SMS → (один раз) согласие на обработку данных.
2. **Главный** — карточка заказа: договор, изделие, финансы (стоимость/предоплата/остаток), этап + прогресс-бар, комментарий менеджера, прогноз услуг, кнопка «Чат с сервисом», телефон сервиса (тап → звонок).
3. **История этапов** — таймлайн прохождения 7 этапов.
4. **Чат** — переписка с сервисом, отправка текста (и фото).
5. **Профиль** — имя, телефон, выход, удаление аккаунта.

Код приложений готов в `apps/android/` и `apps/ios/`. Их сборка и публикация — см. `docs/mobile-build-and-publish.md` (нужен Mac для iOS, Android Studio для Android, аккаунты разработчика).

6. Как запустить локально (для демо/разработки)

Нужен установленный Docker. По шагам:

1. Поднять инфраструктуру (база, Redis, хранилище файлов):

...

```
pnpm dev:infra
```

...

1. Применить миграции базы (один раз):

...

```
pnpm --filter @vittoria/api prisma:migrate:deploy
```

...

1. Запустить сервер:

...

```
pnpm dev:api
```

...

(API поднимется на `http://localhost:3000`)

- 1. Запустить веб-панель:

...

```
pnpm dev:admin
```

...

(откроется на `http://localhost:5173`)

- 1. Войти: телефон администратора, затем код. В режиме разработки **код печатается в консоль сервера** (строка `[DEV-SMS] ... text="VITTORIA HOME: 1234"`).

Тестовый администратор уже заведён: телефон `+79990000000`.

7. Боевой режим: что включить

Что	Как включить	Статус
Реальный amoCRM	Токен в <code>apps/api/.env</code> (<code>AMOCRM_ACCESS_TOKEN</code>) + <code>AMOCRM_CLIENT_MODE=http</code>	✔️ подключено
Реальные SMS (SMSC.ru)	В <code>.env</code> : <code>SMS_PROVIDER_MODE=smcsc</code> + <code>SMSC_LOGIN</code> + <code>SMSC_PASSWORD</code> (+ <code>SMSC_SENDER</code>)	⌚ ждём логин/пароль SMSC
Push-уведомления	<code>PUSH_PROVIDER_MODE=real</code> + ключи FCM (Android) и APNs (iOS)	⌚ для мобильного релиза
Входящие вебхуки amoCRM	Нужен публичный адрес сервера (деплой/домен) — amoCRM не достигнет до <code>localhost</code>	⌚ этап деплоя

Секреты живут только в `apps/api/.env` (в Git не попадают). Для боевого сервера их хранят в защищённом хранилище.

8. amoCRM: воронка и поля (уже настроено)

Воронка «VITTORIA HOME» (создана, id 10959102), 7 этапов в порядке:

- 1. Подготовка для производства
- 2. Детализовка
- 3. Поступление материалов на склад
- 4. Производство изделия

5. Передача готового изделия на склад
6. Проверка комплектности товара
7. Готовность к передаче клиенту

Менеджеры двигают сделку по этим этапам — приложение показывает клиенту соответствующий этап.

Служебные поля сделки (созданы, только для нашего сервиса, существующие поля не трогаются):

`vittoria_stage`, `vittoria_progress`, `vittoria_admin_comment`, `vittoria_prepayment`,
`vittoria_partner_user_id`, `vittoria_partner_services`.

Если воронку/поля пересоздать — нужно обновить их ID в конфигурации (скрипт `scripts/amocrm-create-pipeline.py` помогает получить ID воронки и этапов).

9. Где что лежит (структура проекта)

- `apps/api/` — сервер (NestJS).
- `apps/admin/` — веб-панель (React).
- `apps/android/` — Android-приложение (Kotlin).
- `apps/ios/` — iOS-приложение (Swift).
- `docs/` — документация (этот файл, контракт API, гайд по сборке мобильных, спецификации/планы в `docs/superpowers/`).
- `scripts/` — служебные скрипты (настройка amoCRM).
- `infra/` — Docker-окружение для разработки.

10. Что ещё предстоит (для полного запуска)

1. **SMSC.ru** — прислать логин/пароль → включу реальные SMS.
2. **Мобильная сборка** — на Mac (iOS) и в Android Studio (Android): собрать, проверить по сценариям, опубликовать в App Store и Google Play (нужны аккаунты разработчика).
3. **Деплой сервера** — выложить backend на сервер с HTTPS-доменом; тогда заработают входящие вебхуки amoCRM (мгновенная синхронизация) и приложения будут ходить на боевой адрес.
4. **Push-ключи** — FCM (Android) и APNs (iOS) для уведомлений в мобильных.

Готов сопровождать каждый из этих шагов.

Инструкция для менеджеров

VITTORIA HOME — инструкция для менеджеров

Как вести клиента, чтобы он в приложении видел актуальный статус заказа. Коротко и по делу.

Главный принцип

Вы работаете **как обычно — в amoCRM**. Приложение клиента само подхватывает изменения. Ваша задача — вовремя двигать сделку по этапам воронки «**VITTORIA HOME**» и заполнять несколько полей.

Вы двигаете сделку по этапу в amoCRM → клиент в приложении сразу видит этот этап и % готовности

Шаг 1. Завести клиента в воронку «VITTORIA HOME»

- Когда сделка переходит в производство, переместите её в воронку «**VITTORIA HOME**» (или создавайте такие сделки сразу в ней).
- В сделке должен быть **контакт с телефоном** — по этому телефону клиент входит в приложение. Без телефона приложение не сможет показать заказ.

Шаг 2. Двигать сделку по этапам (это и есть «статус для клиента»)

Этапы воронки «VITTORIA HOME» — это то, что видит клиент:

1. Подготовка для производства
2. Детализовка
3. Поступление материалов на склад
4. Производство изделия
5. Передача готового изделия на склад
6. Проверка комплектности товара
7. Готовность к передаче клиенту

Просто перетаскивайте сделку на нужный этап по мере продвижения заказа — клиент сразу увидит обновление и получит уведомление.

Шаг 3. Заполнять служебные поля (что ещё видит клиент)

В карточке сделки есть поля для приложения:

- **vittoria_progress** — процент готовности (0–100). Показывается клиенту полосой прогресса.
- **vittoria_admin_comment** — короткий комментарий клиенту (например, «Ваш заказ на финальной проверке»).
- **vittoria_prepayment** — сумма предоплаты (для блока финансов).
- **vittoria_partner_user_id, vittoria_partner_services** — для партнёрских услуг (заполняется при необходимости).

Этап берётся из положения сделки в воронке, а процент/комментарий — из этих полей. Заполняйте их, чтобы клиенту было понятнее.

Шаг 4. Отвечать клиенту в чате

- Клиент может написать вопрос прямо из приложения.
- Сообщения видны в **веб-панели** → **раздел «Чат»**: слева список диалогов (с пометкой непрочитанных), справа переписка. Отвечайте текстом — клиент получит уведомление.
- Среднее ожидаемое время ответа в приложении заявлено как «в течение 2 часов» — старайтесь не задерживать.

Что видит клиент в приложении

- Номер договора и изделие.
- Финансы: стоимость, предоплата, остаток.
- Текущий этап + процент готовности.
- Ваш комментарий (если заполнен).
- Кнопку чата и телефон сервиса.

Частые вопросы

- **Клиент не может войти.** Проверьте, что его телефон указан в контакте сделки (формат +7xxxxxxxxxx).
- **Клиент не видит обновления.** Убедитесь, что сделка в воронке «VITTORIA HOME» и переведена на нужный этап.
- **Нужно срочно сообщить клиенту.** Напишите в чат из веб-панели или заполните комментарий в сделке.

Памятка

- Двигаете сделку по этапам «VITTORIA HOME» → клиент видит статус.
- Заполняете процент и комментарий → клиенту понятнее.
- Отвечаете в чате из веб-панели → клиент спокоен.

Деплой на сервер

VITTORIA HOME — деплой на сервер (чтобы работало постоянно)

Подробный пошаговый рецепт: как поднять backend + веб-панель на сервере так, чтобы они работали круглосуточно, сами перезапускались и обновлялись. Рассчитано на одного человека без опыта DevOps (можно пройти со мной по шагам).

Готовые конфиги уже в репозитории:

- `apps/api/Dockerfile` — образ сервера (проверен сборкой).
- `apps/admin/Dockerfile` + `apps/admin/nginx.conf` — образ веб-панели.
- `infra/docker-compose.prod.yml` — весь стек одной командой.
- `infra/Caddyfile` — авто-HTTPS + маршрутизация по доменам.

Что получится в итоге

```
Интернет —HTTPS—> Caddy —┬─> api.BASH-домен —> контейнер API (NestJS)
                        └─> admin.BASH-домен —> контейнер веб-панели
                              + PostgreSQL + Redis + MinIO (хранилище фото)
```

Всё крутится в Docker на одном сервере, с автоперезапуском. Caddy сам получает и продлевает бесплатные TLS-сертификаты.

Что понадобится (купить/завести)

1. **Сервер (VPS)** — Ubuntu 22.04, минимум 2 ГБ RAM (лучше 4). Подойдут Yandex Cloud, Timeweb, Selectel, Reg.ru и т.п. Для 152-ФЗ — российский провайдер.
2. **Домен** (например `vittoria.app`) и возможность создавать поддомены.
3. **Доступ по SSH** к серверу (логин/пароль или ключ — даёт провайдер).

Рекомендация для России/152-ФЗ: Yandex Cloud (Managed PostgreSQL + Redis + Object Storage). Но для старта проще и дешевле — один VPS по этому рецепту.

Часть 1. Деплой backend + веб-панели (по шагам)

Шаг 1. Арендовать сервер

Закажите VPS с Ubuntu 22.04. Получите его **IP-адрес** и доступ по SSH. Подключитесь:

```
ssh root@IP-АДРЕС-СЕРВЕРА
```

Шаг 2. Настроить домены (DNS)

В панели вашего домена создайте две **A-записи**, обе на IP сервера:

- `api.vittoria.app` → IP
- `admin.vittoria.app` → IP

(Подождите 10–30 мин, пока DNS обновится.)

Шаг 3. Установить Docker на сервер

На сервере выполните:

```
curl -fsSL https://get.docker.com | sh
```

Проверка: `docker --version` и `docker compose version`.

Шаг 4. Загрузить проект на сервер

```
apt-get install -y git
git clone <URL-вашего-репозитория> /opt/vittoria
cd /opt/vittoria
```

(URL репозитория — из GitHub, кнопка Code → HTTPS.)

Шаг 5. Заполнить секреты `infra/.env.prod`

Создайте файл `infra/.env.prod` (он не в Git). Пример — замените значения на свои:

```
# --- база/хранилище (для контейнеров) ---
POSTGRES_USER=vittoria
POSTGRES_PASSWORD=<придумайте_длинный_пароль>
POSTGRES_DB=vittoria
S3_ACCESS_KEY=<придумайте_ключ>
S3_SECRET_KEY=<придумайте_секрет>

# --- адрес API для сборки веб-панели ---
ADMIN_API_BASE_URL=https://api.vittoria.app/api/v1

# --- конфиг сервера (API) ---
NODE_ENV=production
PORT=3000
```

```
DATABASE_URL=postgresql://vittoria:<тот_же_пароль>@postgres:5432/vittoria
REDIS_URL=redis://redis:6379
JWT_SECRET=<случайная_строка_минимум_32_символа>
JWT_ACCESS_TTL_SEC=900
JWT_REFRESH_TTL_SEC=2592000
OTP_TTL_SEC=300
OTP_MAX_ATTEMPTS=5
OTP_REQUEST_RATE_LIMIT_PER_MIN=1
CORS_ORIGINS=https://admin.vittoria.app

# --- amoCRM (уже есть) ---
AMOCRM_BASE_URL=https://vittoriaamo.amocrm.ru
AMOCRM_CLIENT_MODE=http
AMOCRM_ACCESS_TOKEN=<долгосрочный_токен>
AMOCRM_WEBHOOK_SECRET=<случайная_строка_минимум_16>

# --- хранилище фото (MinIO в этом же стеке) ---
S3_ENDPOINT=http://minio:9000
S3_REGION=ru-central1
S3_BUCKET=vittoria-chat

# --- SMS (когда будут доступы SMSC.ru) ---
SMS_PROVIDER_MODE=smc
SMSC_LOGIN=<логин>
SMSC_PASSWORD=<пароль>
SMSC_SENDER=<имя_отправителя_если_есть>

# --- push (для мобильных, позже) ---
PUSH_PROVIDER_MODE=dev
```

Пока нет SMSC — поставьте `SMS_PROVIDER_MODE=dev` (код будет в логах сервера, не в SMS).
Пока нет push-ключей — `PUSH_PROVIDER_MODE=dev`.

Шаг 6. Прописать свои домены в Caddy

Откройте `infra/Caddyfile` и замените `api.vittoria.app` / `admin.vittoria.app` на ваши поддомены.

Шаг 7. Запустить весь стек

```
docker compose -f infra/docker-compose.prod.yml --env-file infra/.env.prod up -d --build
```

Это соберёт образы, поднимет PostgreSQL, Redis, MinIO, API, веб-панель и Caddy. **Миграции БД применяются автоматически** при старте API. Caddy сам выпустит HTTPS-сертификаты для ваших доменов.

Шаг 8. Проверить, что работает

- `https://api.vittoria.app/api/v1/healthz` → `{"status":"ok"}`.
- Откройте `https://admin.vittoria.app` → форма входа.
- Заведите администратора (одноразово), подключившись к БД контейнера:

...

```
docker compose -f infra/docker-compose.prod.yml exec postgres \
```

```
psql -U vittoria -d vittoria -c \
```

```
"INSERT INTO users (id, phone, role, created_at, updated_at) VALUES (gen_random_uuid(),
'+7XXXXXXXXXX', 'admin', now(), now()) ON CONFLICT (phone) DO UPDATE SET role='admin';"
```

...

- Войдите по этому телефону (код придёт SMS, если включён SMSC; иначе — в логах:
`docker compose -f infra/docker-compose.prod.yml logs api | grep DEV-SMS`).

Шаг 9. Включить мгновенную синхронизацию amoCRM (вебхук)

В amoCRM → Настройки → Интеграции → ваша интеграция → раздел вебхуков: добавьте адрес

```
https://api.vittoria.app/api/v1/amocrm/webhooks
```

Теперь изменения сделок будут прилетать мгновенно (без него работает запасная синхронизация раз в 15 минут).

Часть 2. Чтобы работало ПОСТОЯННО

- **Автоперезапуск:** во всех контейнерах стоит `restart: always` — при сбое или перезагрузке сервера они поднимаются сами. Сервер должен быть всегда включён (VPS работает 24/7).
- **Проверка статуса:** `docker compose -f infra/docker-compose.prod.yml ps` (все должны быть `running`).
- **Логи:** `docker compose -f infra/docker-compose.prod.yml logs -f api` (или `admin`, `caddy`).
- **Обновление версии** (после изменений в коде):

...

```
cd /opt/vittoria && git pull
```

```
docker compose -f infra/docker-compose.prod.yml --env-file infra/.env.prod up -d --build
```

```
'''
```

- **Бэкап базы** (рекомендуется ежедневно, cron):

```
'''
```

```
docker compose -f infra/docker-compose.prod.yml exec -T postgres \
```

```
pg_dump -U vittoria vittoria | gzip > /opt/backups/db-$(date +%F).sql.gz
```

```
'''
```

- **Файрвол:** откройте только порты 80 и 443 (Caddy). Базу/Redis/MinIO наружу не публикуйте.
- **Мониторинг (по желанию):** подключить Sentry (ошибки) и UptimeRobot (пинг `/healthz`).

Для повышенной надёжности позже можно вынести PostgreSQL и Redis в управляемые сервисы (Managed) с бэкапами и репликами, а фото — в Object Storage (S3) — тогда сервер становится «одноразовым», а данные защищены отдельно.

Часть 3. Мобильные приложения

Чтобы люди скачивали приложение из Google Play и App Store — регистрация аккаунтов, оплата и публикация описаны отдельно: `docs/store-accounts-and-publishing.md`. Перед сборкой укажите боевой адрес API (`https://api.vittoria.app/api/v1`).

Порядок «под ключ»

1. Сервер + домены + `docker compose up` (Часть 1) — уже можно: amoCRM-токен есть.
2. Добавить SMSC-доступы в `.env.prod` → реальные SMS.
3. Прописать вебхук amoCRM (Шаг 9).
4. Собрать и опубликовать мобильные приложения (Часть 3) + push-ключи.
5. Closed beta → правки → публичный релиз.

Готов пройти любой шаг вместе.

Регистрация и публикация в магазинах

Регистрация в Google Play и App Store: аккаунт, оплата, публикация

Подробно и по шагам: как завести аккаунты разработчика, оплатить и выложить приложение, чтобы люди могли его скачать. Рассчитано на новичка.

Техническая сборка приложений (как собрать файл для загрузки) описана в `docs/mobile-build-and-publish.md`. Здесь — про **аккаунты, оплату и саму публикацию**.

Что понадобится заранее:

- **Иконка** приложения (1024×1024 px) и несколько **скриншотов** экранов.
- **Описание** приложения (краткое и полное) на русском.
- **Политика конфиденциальности** — страница в интернете с текстом (обязательна, т.к. приложение работает с телефоном/именем). Можно разместить на простом сайте или в Notion с публичной ссылкой.
- Банковская карта для оплаты сборов.

A. Google Play (Android) — разовый сбор \$25

A1. Завести аккаунт разработчика

1. Нужен обычный аккаунт **Google** (Gmail). Если нет — создайте на accounts.google.com.
2. Откройте **play.google.com/console**, войдите этим аккаунтом.
3. Выберите тип: **Личный** (физлицо) или **Организация** (юрлицо — нужны данные компании).
4. Заполните профиль разработчика: имя/название (его увидят пользователи), контакты.
5. **Оплатите регистрационный сбор \$25** (разово, навсегда) картой.
6. Пройдите **верификацию личности/организации** (Google попросит документы; проверка от нескольких часов до пары дней). Для организаций может потребоваться номер D-U-N-S.

A2. Создать приложение

1. В Play Console: **Создать приложение**.
2. Название: **VITTORIA HOME**, язык по умолчанию — русский, тип — «Приложение», бесплатное.
3. Подтвердите декларации (правила, экспортное законодательство).

A3. Заполнить карточку и анкеты (обязательное)

В разделе «Создание контента»/«Информация о приложении»:

1. **Краткое и полное описание**, иконка (512×512 в карточке), **скриншоты** телефона (мин. 2).
2. **Категория** (например «Бизнес» или «Стиль жизни»), контактные данные.
3. **Политика конфиденциальности** — вставьте ссылку на вашу страницу.
4. **Возрастной рейтинг** — пройдите анкету.
5. **Безопасность данных (Data safety)** — укажите, что собираете телефон и имя (для авторизации и сервиса).
6. **Доступ к приложению** — если вход только по приглашению/договору, дайте Google тестовый телефон + опишите, как войти (иначе ревью не сможет проверить).

A4. Загрузить приложение и выпустить

1. Соберите подписанный **AAB** (см. `docs/mobile-build-and-publish.md`, раздел Android). **Сохраните ключ подписи (keystore) и пароли** — без них нельзя выпускать обновления.
2. Сначала трек **Внутреннее тестирование (Internal testing)**: создайте релиз, загрузите `.aab`, добавьте себя в тестировщики, проверьте установку и вход.
3. Когда всё ок — трек **Production (Открытое тестирование/Релиз)**: создайте релиз, загрузите AAB, **Отправить на проверку**.
4. Проверка Google — обычно несколько часов–дней. После одобрения приложение появляется в Google Play.

B. App Store (iOS) — подписка \$99/год, нужен Mac

Для iOS обязательны **компьютер Mac** (для сборки/загрузки) и платный **Apple Developer Program**.

B1. Создать Apple ID и вступить в Apple Developer Program

1. Создайте **Apple ID** (appleid.apple.com), включите двухфакторную аутентификацию.
2. Откройте **developer.apple.com** → Account → **Enroll** (вступить в Apple Developer Program).
3. Тип: **Individual** (физлицо) или **Organization** (юрлицо — потребуется D-U-N-S номер компании, его получают бесплатно, но может занять время).
4. **Оплатите \$99/год** картой.
5. Дождитесь активации членства (от нескольких часов до 1–2 дней; для организаций дольше из-за верификации).

B2. Создать приложение в App Store Connect

1. Откройте **appstoreconnect.apple.com** → «Мои приложения» → + → Новое приложение.
2. Платформа iOS, название **VITTORIA HOME**, основной язык — русский.
3. **Bundle ID**: `app.vittoria.client` (создаётся в разделе Certificates, Identifiers & Profiles, либо выбирается при создании).
4. SKU — любой внутренний код (например `vittoria-home`).

B3. Собрать и загрузить сборку (на Mac)

1. На Mac: `cd apps/ios && xcodegen generate`, откройте `VittoriaClient.xcodeproj` в Xcode.
2. В Signing выберите вашу команду (Apple Developer). Укажите боевой адрес API.
3. `Product` → `Archive` → в Organizer: **Distribute App** → **App Store Connect** → **Upload**.
Сборка появится в App Store Connect через несколько минут.

B4. Заполнить карточку и анкеты, отправить на ревью

1. **Иконка 1024×1024**, **скриншоты** нужных размеров (для разных диагоналей iPhone), **описание**, ключевые слова.
2. **Политика конфиденциальности** (ссылка) и анкета **App Privacy** — укажите сбор телефона/имени.
3. **Возрастной рейтинг**.
4. Выберите загруженную сборку → **Submit for Review**.
5. Ревью Apple — обычно 1–3 дня. Часто просят **демо-доступ**: дайте тестовый телефон + объясните, как войти (вход по SMS-коду). После одобрения — публикация в App Store.

Частые причины отказов (чтобы пройти с первого раза)

- Нет рабочего **демо-доступа** для проверяющих (вход по приглашению/договору) — обязательно дайте тестовый аккаунт.
- Нет/недоступна **политика конфиденциальности** — разместите рабочую ссылку.
- Несоответствие **анкеты приватности** реальному сбору данных — указывайте честно (телефон, имя).
- iOS: приложение «просто веб-обёртка» — у нас полноценный нативный клиент, это ок.

Итоговый чек-лист

- [] Иконка 1024×1024 + скриншоты + описания.
- [] Страница политики конфиденциальности (публичная ссылка).
- [] Google Play Developer (\$25, верификация) → AAB → анкеты → на проверку.

- [] Apple Developer (\$99/год) + Mac → Archive/Upload → анкеты → на ревью.
- [] Тестовый телефон для демо-доступа проверяющим.

Готов помочь на каждом шаге (особенно со сборкой AAB и архивом iOS).

Что нужно для размещения: кто что делает

Честно о ролях — почему «выложить за тебя полностью» я не могу и что для этого нужно.

Чего я НЕ могу сделать сам (ограничения платформ)

- **Загрузить приложение в App Store** — это возможно только с **Mac + Xcode** (правило Apple). У меня среда Windows.
- **Собрать/загрузить Android AAB** — нужен Android Studio/SDK, которого в этой среде нет.
- **Войти в твои аккаунты** Apple Developer / Google Play — там личная 2FA-аутентификация; доступ к ним остаётся только у тебя.

Что я ПОДГОТОВЛЮ (и уже подготовил)

- Весь **код** обоих приложений (`apps/ios` , `apps/android`).
- **Конфиги сборки** (XcodeGen `project.yml` для iOS, Gradle для Android), Bundle ID `app.vittoria.client` .
- **Тексты карточек** магазинов (название, краткое/полное описание) — напишу под твой бренд.
- Помогу **по шагам** при сборке и при заполнении карточек; если при сборке возникнут ошибки — пришли их мне, я подскажу правки.

Что нужно ОТ ТЕБЯ (чтобы подготовить и опубликовать)

1. **Аккаунты (создаёшь и оплачиваешь сам):** Apple Developer (\$99/год), Google Play Developer (\$25 разово).
2. **Машина для сборки: Mac с Xcode** — для iOS обязательно; **Android Studio** (Windows/Mac/Linux) — для Android. Это можешь сделать ты или нанятый разработчик; я веду по шагам.
3. **Материалы карточки:**
 - Иконка приложения **1024×1024 px**.
 - **Скриншоты** главных экранов (с реального телефона/эмулятора — появятся после сборки).
 - **Описание** и название (могу составить — нужно твоё «ок»).
 - **Ссылка на политику конфиденциальности** (публичная страница).

1. **Боевой адрес API** — будет после деплоя (`https://api.твой-домен/api/v1`); его зашиваем в сборку.
2. **Тестовый доступ для проверяющих** — телефон + как войти (вход по SMS-коду), иначе ревью не пройдёт.

Как мы это делаем на практике

1. Ты создаёшь и оплачиваешь аккаунты (по инструкции выше).
2. Готовишь Mac (для iOS) / ставишь Android Studio (для Android) — или передаёшь это разработчику.
3. Я даю точные команды сборки и тексты карточек; ты (или разработчик) запускаешь сборку, при ошибках — присылаешь мне, я правлю.
4. Ты загружаешь готовый файл (AAB / архив) в консоль магазина под своим аккаунтом, заполняешь карточку (по моим текстам), отправляешь на проверку.
5. Я на связи на каждом шаге.

Короче: код и тексты — на мне; аккаунты, оплата, Mac/Android Studio и финальная загрузка под твоим логином — на тебе (с моим пошаговым сопровождением).

Сборка и публикация мобильных приложений

Мобильные приложения — сборка и публикация

Исходный код обоих приложений написан, но **на этом компьютере (Windows) он не собирался и не проверялся**. Это нормально: сборка iOS возможна только на Mac, а Android — в Android Studio. Ниже — что нужно и какие шаги, простыми словами.

Код лежит в репозитории:

- **Android** — папка `apps/android/` (Kotlin, Jetpack Compose).
- **iOS** — папка `apps/ios/` (Swift, SwiftUI).

Оба приложения общаются с тем же backend по REST API (контракт — `docs/superpowers/specs/2026-05-29-mobile-client-contract.md`).

Что обязательно понадобится

Для чего	Что нужно	Стоимость
Собрать/опубликовать iOS	Компьютер Mac + программа Xcode	Mac — железо; Xcode бесплатен
Опубликовать в App Store	Аккаунт Apple Developer	\$99 / год
Собрать Android	Android Studio (Windows/Mac/Linux)	бесплатно
Опубликовать в Google Play	Аккаунт Google Play Developer	\$25 один раз
Боевой адрес API	Развёрнутый backend на сервере (HTTPS-домен)	по инфраструктуре

Android — как собрать и опубликовать (пошагово)

А. Сборка и проверка

1. Установить **Android Studio** (последнюю версию) + JDK 17 (идёт в комплекте).
2. Открыть папку `apps/android/` в Android Studio. Дождаться «Gradle Sync» (подтянет зависимости — нужен интернет).
3. Запустить на эмуляторе/телефоне (кнопка ►). В разработке приложение ждёт API на `http://10.0.2.2:3000` (это «localhost» для эмулятора). Прогнать сценарии: вход по SMS → главный экран → чат → профиль. Зафиксировать мелкие правки, если что-то не собралось.

4. Перед боевым релизом поменять адрес API на ваш HTTPS-домен (в сетевом конфиге приложения).

Б. Публикация в Google Play

1. Зарегистрировать **аккаунт Google Play Developer** (play.google.com/console) — разовый взнос **\$25**, верификация личности/организации.
2. Создать **ключ подписи** (keystore): в Android Studio `Build → Generate Signed Bundle/APK → Android App Bundle → Create new...`. **Сохранить keystore и пароли в надёжном месте** — без него нельзя выпускать обновления.
3. Собрать **release AAB**: `Build → Generate Signed Bundle/APK → Android App Bundle`, выбрать keystore, вариант `release`. Получится файл `.aab`.
4. В Google Play Console: **Create app** → название «VITTORIA HOME», язык, тип «App», бесплатное.
5. Заполнить обязательное: иконка (512×512), скриншоты (телефон), краткое и полное описание, **политика конфиденциальности** (URL — нужна из-за работы с ПДн), категория, контакты, анкеты «Content rating» и «Data safety» (указать сбор телефона/имени).
6. Загрузить `.aab` в трек **Internal testing** (для своих) → проверить → затем **Production** → **Submit for review**. Проверка обычно несколько часов-дней.

iOS — как собрать и опубликовать (нужен Mac, пошагово)

А. Сборка и проверка (на Mac)

1. Установить **Xcode** (из App Store) и **XcodeGen**: `brew install xcodegen`.
2. `cd apps/ios && xcodegen generate` — создаст `VittoriaClient.xcodeproj`.
3. Открыть проект в Xcode → в **Signing & Capabilities** выбрать вашу команду (Apple Developer). Запустить на «iOS Simulator». Прогнать сценарии (как у Android). Зафиксировать мелкие правки SDK-версий, если всплывут.
4. Перед релизом поменять адрес API на боевой HTTPS-домен.

Б. Публикация в App Store

1. Оформить **Apple Developer Program** (developer.apple.com) — **\$99/год**, верификация.
2. В **App Store Connect** (appstoreconnect.apple.com) создать приложение: Bundle ID `app.vittoria.client`, название «VITTORIA HOME».
3. В Xcode выбрать устройство «Any iOS Device», затем `Product → Archive`. В окне Organizer → **Distribute App** → **App Store Connect** → **Upload**.
4. В App Store Connect заполнить карточку: иконка (1024×1024), скриншоты (нужных размеров), описание, ключевые слова, возрастной рейтинг, **политика конфиденциальности** (URL) и анкета «App Privacy» (сбор телефона/имени).
5. Выбрать загруженный билд → **Submit for Review**. Ревью Apple обычно 1–3 дня (бывает дольше). Часто просят демо-доступ — дать тестовый телефон + код.

Важно про «всё должно работать»

- **Что уже работает и можно показать сейчас:** backend + веб-панель (admin и кабинет партнёра) — они запускаются на этом ПК, вы их видели.
- **Мобильные приложения:** код готов, но «работают» они станут после первой сборки на Mac (iOS) / в Android Studio (Android). Почти наверняка при первой сборке всплывут мелкие правки (версии библиотек, подпись) — это нормальный этап для любого мобильного проекта. После сборки нужно прогнать по сценариям: вход по SMS → главный экран с заказом → чат → профиль.
- **amoCRM:**  уже подключён (реальный режим `http`, токен в `.env`, воронка «VITTORIA HOME» и служебные поля созданы). Входящие вебхуки заработают после деплоя на публичный домен.
- **Реальные SMS:**  нужны логин/пароль SMSC.ru → переключение SMS в режим `smcsc`. Сейчас код подтверждения печатается в консоль сервера (dev-режим).

Рекомендованный следующий шаг

Когда появится Mac и аккаунты разработчика — открыть оба проекта, собрать, прогнать сценарии и зафиксировать список мелких правок. Я помогу пройти этот этап и довести до зелёной сборки.